

Programming

Lecture 05 — Strings

Text as a sequence: indexing, slicing, and methods

Dr. Ramon Winterhalder

Human-Centered Artificial Intelligence · A.Y. 2026/27

UNIVERSITÀ DEGLI STUDI DI MILANO

INFN MILANO

Where we left off

Lecture 04, in four lines

while repeats until something changes
Set up, condition, body, change.

for repeats a known number of times
The counter is made for you.

The accumulator
Declared outside, updated inside, read after.

A flag remembers what a loop found
Set it True, let the loop disprove it.

You have been printing text since week one without ever looking inside it. Today you take it apart — one character at a time.

Everything you learn today about [ ] and : works on lists next week, unchanged. This is not a detour — it is half of lecture 6, learned early.

By the end of today you can…

- pick out any character or any piece of a string;
- say what "strings are immutable" means, and what it forces you to do;
- use the string methods you will actually need, and look up the rest;
- loop over text and count things in it;
- split a sentence into words.

One promise comes due today: sheet 3, B1 number 6 — comparing "Python" with "python" — where the answer was "you need .lower(), and we meet it in lecture 5".

A

Strings as sequences

Indexing · slicing · immutability · looping

A string is a sequence of characters

Each one has a number

	P	y	t	h	o	n
index	0	1	2	3	4	5
from the end	−6	−5	−4	−3	−2	−1

```
s = "Python"

s[0]    ->  "P"          s[-1]  ->  "n"      the last one
s[3]    ->  "h"          s[-2]  ->  "o"      the one before it
len(s) ->  6             s[6]   ->  IndexError
```

Counting starts at 0, so the last index is len(s) - 1. Negative numbers count back from the end, which saves you writing s[len(s) - 1].

Slicing

A piece, not a character

```
s = "Programming"

s[0:4]    ->  "Prog"        from 0, stop before 4
s[4:7]    ->  "ram"
s[:4]     ->  "Prog"        leave out the start: from the beginning
s[4:]     ->  "ramming"     leave out the end: to the end
s[:]      ->  "Programming" the whole thing
s[-4:]    ->  "ming"        the last four
```

Stop before, again. s[0:4] gives four characters: 0, 1, 2, 3 — exactly like range(0, 4).

A slice always gives you a string back, even a one-character slice: s[0:1] is "P", and so is s[0].

Slicing with a step

And the famous one

```
s = "Programming"

s[0:8:2]   ->  "Porm"       every second, from 0 up to 8
s[::2]     ->  "Pormig"     every second, whole string
s[::-1]    ->  "gnimmargorP"    backwards
```

- s[start:stop:step] — the same three numbers as range().
- A negative step walks backwards, so s[::-1] reverses a string.
- You will meet s[::-1] constantly. It is worth recognising on sight.

Do not go hunting for a reverse() function for strings. There is none, and there does not need to be.

Old friends, new type

You have used all of these already

	means	example	result
len(s)	how many characters	len("Python")	6
+	join two strings	"Pro" + "gram"	"Program"
*	repeat	"ab" * 3	"ababab"
in	is it inside?	"gram" in "Programming"	True
not in	is it absent?	"z" not in "Python"	True
== !=	same characters?	"Yes" == "yes"	False
< >	dictionary order	"apple" < "banana"	True

Comparison is character by character, using the computer's code for each one. Capitals come before lower case, so "Zoo" < "apple" is True — which is almost never what a human means by alphabetical order.

That is the answer to sheet 3, B1 number 6. The fix is on the next block: compare s.lower() instead.

Strings are immutable

You cannot change one. Ever.

What gives you an error

```
s = "cat"
s[0] = "b"
```

```
TypeError: 'str' object does not
support item assignment
```

What you do instead

```
s = "cat"
s = "b" + s[1:]      # "bat"
```

Build a NEW string and attach the old name to it. The original is untouched — there is simply no name pointing at it any more.

Every operation on a string gives you a new string. Nothing you do modifies the one you started with.

This is not a limitation to work around — it is why you can pass a string anywhere without worrying that something will change it behind your back. Lists, next week, are the opposite.

Looping over text

A string is something you can iterate

```
for ch in "Python":
    print(ch)
```

```
P
y
t
h
o
n
```

Counting with an accumulator

```
text = "programming"
vowels = 0

for ch in text:
    if ch in "aeiou":
        vowels += 1

print(vowels)           # 3
```

The accumulator from last week, over characters instead of numbers. Same shape, new thing to walk through.

Two ways to visit every character: ***for ch in s*** gives you the characters, ***for i in range(len(s))*** gives you the positions. Use the first unless you need the index.

Escape sequences

Characters you cannot type

written	means	
\n	a new line	print("a\nb") prints two lines
\t	a tab	rough column alignment
\"	a double quote	inside a double-quoted string
\'	a single quote	inside a single-quoted string
\\	a backslash	because \ starts an escape

```
print("She said \"hello\" and left")
print('She said "hello" and left')        # easier: swap the quotes
```

Sheet 1, B3 number 6 was exactly this. The answer then was "swap the quotes", and it still is — an escape is what you reach for when both kinds appear in one string.

Worked example

Is it a palindrome?

Pass 1 — the naive version

```
text = input("A word: ")

if text == text[::-1]:
    print("palindrome")
else:
    print("not a palindrome")
```

Works on "level". Fails on "Level".

Pass 2 — ignore capitals

```
text = text.lower()

if text == text[::-1]:
```

Pass 3 — ignore everything else

```
clean = ""
for ch in text.lower():
    if ch in "abcdefghijklmnopqrstuvwxyz":
        clean = clean + ch

if clean == clean[::-1]:
```

Pass 3 is the accumulator again — building a string instead of a number.
Try it on "A man, a plan, a canal: Panama".

Exercises — Block A

Sheet 05, part A · 40 minutes · on paper first

- Give the value of twelve slices of one word.
- Say which of six expressions are True.
- Write the slice that extracts a given piece.
- Explain why one program raises TypeError, and fix it.
- Count something in a string with a loop.

Write the index numbers under the word before you answer anything. Every slicing mistake is an off-by-one, and you cannot see an off-by-one without the numbers in front of you.

B

String methods

The dot · the useful ones · split and join

A new piece of syntax: the dot

The dot

```
text = "  Hello World  "

text.lower()      # ->  "  hello world  "
text.strip()      # ->  "Hello World"
text.upper()      # ->  "  HELLO WORLD  "
```

- len(s) is a function: the string goes inside the brackets.
- s.lower() is a METHOD: the string comes first, then a dot, then the name.
- A method belongs to the thing it is attached to. Only strings have .lower().
- The brackets are still there, and still mean "run it now".

The dot is how Python says "belonging to". You will see it everywhere from here on — and in lecture 11 you will write your own.

The ones worth knowing

method	gives back	example on " Hello World "
.lower() .upper()	a new string, case changed	" hello world "
.strip()	a new string, no spaces at the ends	"Hello World"
.replace(a, b)	a new string, every a swapped for b	.replace(“l","L") -> “ HeLLo WorLd "
.count(x)	a number: how many times	.count("l") -> 3
.find(x)	the index of the first x, or -1	.find("World") -> 8
.startswith(x) .endswith(x)	True or False	.strip().startswith(“He") -> True
.split()	the words, as a list	["Hello", "World"]

You are not expected to memorize this. You are expected to know that string methods exist, roughly what is available, and how to look one up.

In a notebook, type text. and press Tab: Python lists everything a string can do. help(str.strip) explains any one of them.

The bug this causes

Immutability, second sighting

Does nothing

```
name = "ada"
name.upper()

print(name)      # "ada"
```

Works

```
name = "ada"
name = name.upper()

print(name)      # "ADA"
```

A string method never changes the string. It hands you a new one, and if you do not catch it, it is gone.

The same is true for `.strip()`, `.replace()`, `.lower()` and every other one. If the method gives something back, you have to put it somewhere.

This is the single most common string bug, and it is completely silent: no error, no warning, and the program carries on with the old value.

split and join

From text to words and back

```
sentence = "the quick brown fox"

words = sentence.split()
# ["the", "quick", "brown", "fox"]

len(words)              # 4

for w in words:
    print(w)
```

- split() with nothing inside splits on spaces.
- split(",") splits on commas — useful for data files, lecture 9.
- What comes back is a LIST. That is next week.
- For today: you can loop over it, and ask for its len().

```
"-".join(words)      ->  "the-quick-brown-fox"
```

join is split backwards. The string you call it on is the glue that goes between the pieces — which reads oddly until you have used it twice.

Worked example

Statistics on a paragraph

```
text = input("Paste some text: ")

print(f"characters: {len(text)}")
print(f"words:      {len(text.split())}")

longest = ""
for w in text.split():
    if len(w) > len(longest):
        longest = w

print(f"longest:    {longest}")
```

Letter frequency

```
low = text.lower()

for letter in
"abcdefghijklmnopqrstuvwxyz":
    n = low.count(letter)
    if n > 0:
        print(letter, n)
```

A loop over the alphabet, asking the text how many of each it holds. Three ideas from three different weeks, in six lines.

The longest-word loop is the running maximum from sheet 4, B7 — with len(w) in place of the number itself. Same shape, third outing.

Exercises — Block B

Sheet 05, part B · 40 minutes · plan in comments first

Write

- Clean up a messy name and print it properly.
- Count the vowels, then the words.
- Check whether a file name ends in .csv.
- Build the initials from a full name.

Read and predict

- Eight method calls: what comes back?
- Four programs, two of which do nothing at all.
- One that runs and gives the wrong count.

In the "does nothing" questions, look for a method call whose result was never caught. That is the bug, every time.

Today in one slide

A string is a sequence

s[0] is the first, s[-1] the last, s[a:b] stops before b.

s[::-1] reverses it

Same three numbers as range, including the step.

Strings are immutable

You never change one. You build a new one.

Methods use a dot

s.lower(), s.strip(), s.split() —
and they give back a new string.

Catch what a method returns

name.upper() alone does nothing at all, silently.

Comparison is code order

"Zoo" < "apple". Use .lower() before comparing.

Before next time

- Finish sheet 05. Solutions go up after we discuss them.
- In a notebook, type a string, then a dot, then Tab. Read what comes up.
- Read chapter 3.1 of Sedgewick, Wayne & Dondero.
- Bring the laptop.

Next: Lecture 06 — lists. The same brackets and the same colons, on something you are allowed to change. That one difference causes all the trouble.